Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій

Кафедра САП



Методичні вказівки до лабораторної роботи № 4

з дисципліни: “Організація баз даних та знань”
на тему:
“**Написання запитів SELECT. Ч. 1**”

Затверджено на засіданні кафедри
“*Системи автоматизованого проєктування*”
Протокол № 1 від 25.08.2026 р.

Львів-2026 р.

НАПИСАННЯ ЗАПИТІВ SELECT. Ч. 1. Методичні вказівки до лабораторної роботи № 4 з курсу “Організація баз даних та знань” для студентів базового напрямку “Комп’ютерні науки” / Укл. О. М. Станкевич. – Львів: НУЛП, 2026. – 31 с.

Укладач: О. М. Станкевич

1. МЕТА РОБОТИ

Мета роботи – отримання практичних навичок створення запитів на вибірку, сортування, фільтрування та групування даних засобами SQL.

2. ТЕОРЕТИЧНІ ВІДОМОСТІ

Традиційно, мову SQL поділяють на три частини: мова визначення даних DDL (data definition language), мова маніпулювання даними DML (data manipulation language) і мова управління даними DCL (data control language). До DDL відносять оператори роботи зі схемою бази даних, до DML – оператори вибірки і зміни даних таблиць, які не змінюють схему. Мова DCL складається, в основному, з операторів управління транзакціями і призначенням прав доступу. Основним оператором мови DML є оператор SELECT – вибірка даних із таблиць.

Запис вибраних полів у файл. Оператор **SELECT** можна задати у формі:

SELECT <список полів> **INTO OUTFILE** ‘ім’я_файла’ **FROM** <ім’я таблиці>.

Для використання цієї форми в операторі **SELECT** необхідні привілегії **FILE**. Приклад запису двох полів таблиці student у зовнішній файл:

SELECT kod_stud, fam **INTO OUTFILE** ‘C://result.txt’ FROM student;

Вибірка всіх полів і записів з таблиці.

Формат оператора **SELECT**:

SELECT * **FROM** <ім’я_таблиці>

Вибірка унікальних значень – оператор DISTINCT.

За допомогою оператора **DISTINCT** можна вибрати унікальні дані за певними стовпцями.

!Важливо. Під час роботи з базою даних необхідно вказати після оператора її створення оператор

USE productsdb;

Можна цей оператор опустити. Тоді необхідно активувати базу даних productsdb у розділі **SCHEMAS** подвійним кліком миші на її назві, і вона виділиться жирним шрифтом.

Наприклад, різні товари можуть мати одних і тих же виробників, і, припустимо, маємо таблицю товарів:

```
1  create database productdb;
2  use productdb;
3
4  create table product(
5  id int auto_increment primary key,
6  product_name varchar(30) not null,
7  manufacturer varchar(30) not null,
8  product_count int default 0,
9  price decimal not null);
10
11 insert into product (product_name, manufacturer, product_count, price)
12 values
13 ('iphone X', 'Apple', 2, 76512),
14 ('iphone 8', 'Apple', 2, 51834),
15 ('iphone 7', 'Apple', 5, 42086),
16 ('Galaxy S9', 'Samsung', 2, 56532),
17 ('Galaxy S8', 'Samsung', 1, 46066),
18 ('Honor 10', 'Huawei', 2, 26565),
19 ('Nokia 8', 'HMD Global', 6, 38855),
20 ('iphone X1', 'Apple', 7, 56045),
21 ('iphone 81', 'Apple', 7, 31032),
```

Виберемо всіх виробників:

```
42 •   select manufacturer from product;
43
```

Result Grid | Filter Rows: | Export:

manufacturer
Apple
Apple
Apple
Samsung
Samsung
Huawei
HMD Global
Apple
Apple
Apple
Samsung

Однак під час такого запиту виробники повторюються. Тепер застосуємо оператор **DISTINCT** для вибірки унікальних значень:

```
42 •   select distinct manufacturer from product;
43
```

Result Grid | Filter Rows: | Export: | Wr

manufacturer
Apple
Samsung
Huawei
HMD Global

Також можна задавати вибірку унікальних значень за декількома стовпцями:

```
•   select distinct manufacturer, product_count from products;
```

У цьому випадку для вибірки використаємо стовпці manufacturer і product_count. Стовпці, які мають значення, які повторюються, у вибірці будуть виведені оди раз:

```
42 •   select distinct manufacturer, product_count from product;
43
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

manufacturer	product_count
Apple	2
Apple	5
Samsung	2
Samsung	1
Huawei	2
HMD Global	6

Оператори фільтрації.

Оператор IN. Задає набір значень, які повинні мати стовпці:

WHERE назва_стовпця [NOT] IN (значення 1 [, значення 2, …])

Вираз в дужках після IN задає набір значень. Цей набір можна обчислювати динамічно, на основі ще одного запиту, або це можуть бути константні значення. *Наприклад*, виберемо товари, в яких виробник або Samsung, або HTC, або Huawei:

```
42 •   select * from product
43     where manufacturer in('Samsung', 'HTC', 'Huawei');
44
```

Result Grid | Filter Rows: | Edit: | Expo

id	product_name	manufacturer	product_count	price
4	Galaxy S9	Samsung	2	56532
5	Galaxy S8	Samsung	1	46066
6	Honor 10	Huawei	2	26565
11	Galaxy S91	Samsung	7	36035
12	Galaxy S81	Samsung	6	26044
13	Honor 101	Huawei	7	16081
18	Galaxy S92	Samsung	9	26434

Оператор **NOT**, навпаки, вибирає всі рядки, стовпці яких не мають заданих значень:

```
42 •  select * from product
43    where manufacturer not in('Samsung', 'HTC', 'Huawei');
44
```

Result Grid | Filter Rows: | Edit: | Export/Im

id	product_name	manufacturer	product_count	price
1	iphone X	Apple	2	76512
2	iphone 8	Apple	2	51834
3	iphone 7	Apple	5	42086
7	Nokia 8	HMD Global	6	38855
8	iphone X1	Apple	7	56045

Оператор BETWEEN. Задає діапазон значень за допомогою початкового та кінцевого значення, якому повинен відповідати вираз:

WHERE назва_стовпця [NOT] BETWEEN почат_знач AND кінц_знач;

Наприклад, отримаємо всі товари, у яких ціна від 40000 до 50000 (початкове і кінцеве значення також включаються у діапазон):

```
42 •  select * from product
43    where price between 40000 and 50000;
44
```

Result Grid | Filter Rows: | Edit:

id	product_name	manufacturer	product_count	price
3	iphone 7	Apple	5	42086
5	Galaxy S8	Samsung	1	46066
15	iphone X2	Apple	9	46865
24	iphone 73	Apple	15	46577
26	Galaxy S83	Samsung	11	48112
28	Nokia 83	HMD Global	16	40746
NULL	NULL	NULL	NULL	NULL

Якщо потрібно, навпаки, вибрати ті рядки, значення яких не належать заданому діапазону, то додається оператор **NOT**:

```
42 •   select * from product
43     where price not  between 40000 and 50000;
44
```

Result Grid | Filter Rows: | Edit:

id	product_name	manufacturer	product_count	price
1	iphone X	Apple	2	76512
2	iphone 8	Apple	2	51834
4	Galaxy S9	Samsung	2	56532
6	Honor 10	Huawei	2	26565
7	Nokia 8	HMD Global	6	38855
8	iphone X1	Apple	7	56045
9	iphone 81	Apple	7	31032
10	iphone 71	Apple	7	22012

Також можна використати складніші вирази. *Наприклад*, отримаємо товари за сумарною ціною (ціна * кількість):

```
42 •   select * from product
43     where price*product_count not  between 90000 and 150000;
44
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id	product_name	manufacturer	product_count	price
1	iphone X	Apple	2	76512
3	iphone 7	Apple	5	42086
5	Galaxy S8	Samsung	1	46066
6	Honor 10	Huawei	2	26565
7	Nokia 8	HMD Global	6	38855
8	iphone X1	Apple	7	56045
9	iphone 81	Apple	7	31032
10	iphone 71	Apple	7	22012

Оператори LIKE і REGEXP. Задає шаблон рядка, якому повинен відповідати вираз:

WHERE назва_стовпця [NOT] LIKE шаблон_рядка;

Для задання шаблону можна використовувати низку спеціальних символів підстановки:

- %: відповідає будь-якій підстановці, яка може мати будь-яку кількість символів, водночас підрядок може і не містити жодного символу;

  Наприклад, вираз **WHERE product_name LIKE 'Galaxy%'** відповідає таким значенням як “Galaxy Ace 2” або “Galaxy S7”

- _: відповідає будь-якому одиничному символу;

Наприклад, вираз WHERE product_name LIKE 'Galaxy S_' відповідає таким значення як “Galaxy S7” або “Galaxy S8”.

Застосуємо оператор **LIKE**:

```
42 •  select * from product
43    where product_name like 'iphone%';
44
```

Result Grid | Filter Rows: | Edit:

id	product_name	manufacturer	product_count	price
1	iphone X	Apple	2	76512
2	iphone 8	Apple	2	51834
3	iphone 7	Apple	5	42086
8	iphone X1	Apple	7	56045
9	iphone 81	Apple	7	31032
10	iphone 71	Apple	7	22012
15	iphone X2	Apple	9	46865

REGEXP дає змогу задати регулярний вираз, якому повинно відповідати значення стовпця. Тут **REGEXP** представляє собою більш витончений і комплексний спосіб фільтрації, ніж оператор **LIKE**. **REGEXP** має подібний синтаксис:

WHERE назва_стовпця [NOT] REGEXP регулярний_вираз;

Регулярний вираз може мати такі спеціальні символи:

- ^: вказує на початок рядка
- $: вказує на кінець рядка
- .: відповідає будь-якому одиничному символу
- [символи]: відповідає будь-якому одиничному символу з дужок
- [початковий_символ-кінцевий_символ]: відповідає будь-якому одиничному символу із діапазону символів
- |: відокремлює два шаблона рядка, і значення повинно відповідати одному із цих шаблонів.

Приклади REGEXP:

- **WHERE product_name REGEXP 'phone';**: рядок повинен містити "phone", наприклад, phone X, Nokia phone N, iphone
- **WHERE product_name REGEXP '^phone';**: рядок повинен починатися з "phone", наприклад, phone 34, phoneX
- **WHERE product_name REGEXP 'phone$';**: рядок повинен закінчуватись на "phone", наприклад, iphone, Nokia phone
- **WHERE product_name REGEXP 'ipPhone [78]';**: рядок повинен містити або iphone 7 або iphone 8
- **WHERE product_name REGEXP 'iphone [6-8]';**: рядок повинен містити або iphone 6, або iphone 7, або iphone 8.

Наприклад, знайдемо товари, назви яких містять “phone” або “Galaxy”:

```
42 •  select * from product
43    where product_name regexp 'iphone|Galaxy';
44
```

Result Grid | Filter Rows: | Edit:

id	product_name	manufacturer	product_count	price
1	iphone X	Apple	2	76512
2	iphone 8	Apple	2	51834
3	iphone 7	Apple	5	42086
4	Galaxy S9	Samsung	2	56532
5	Galaxy S8	Samsung	1	46066
8	iphone X1	Apple	7	56045
9	iphone 81	Apple	7	31032

Оператор IS NULL.

```
42 •  select * from product
43    where product_count is null;
```

За допомогою оператора **NOT** можна, навпаки, вибрати рядки, стовпці яких не мають значення **NULL**:

```
42 •  select * from product
43    where product_count is not null;
44
```

Result Grid | Filter Rows: | Edit:

id	product_name	manufacturer	product_count	price
1	iphone X	Apple	2	76512
2	iphone 8	Apple	2	51834
3	iphone 7	Apple	5	42086
4	Galaxy S9	Samsung	2	56532
5	Galaxy S8	Samsung	1	46066
6	Honor 10	Huawei	2	26565

Сортування ORDER BY. Сортує значення за одним чи декількома стовпцями. Наприклад, упорядкуємо вибірку із таблиці product за стовпцем price:

```
42 •  select * from product
43    order by price;
44
```

Result Grid | Filter Rows: | Edit:

id	product_name	manufacturer	product_count	price
20	Honor 102	Huawei	9	12121
17	iphone 72	Apple	12	12567
13	Honor 101	Huawei	7	16081
19	Galaxy S82	Samsung	8	16455
14	Nokia 81	HMD Global	11	18544
21	Nokia 82	HMD Global	13	18923
16	iphone 82	Apple	9	21834
10	iphone 71	Apple	7	22012
12	Galaxy S81	Samsung	6	26044
18	Galaxy S92	Samsung	9	26434
6	Honor 10	Huawei	2	26565

Також можна упорядковувати дані за псевдонімом стовпця, який задається за допомогою оператора **AS**.

```
42 •  select product_name, price*product_count as totalsum
43    from product
44    order by totalsum;
45
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

product_name	totalsum
Galaxy S8	46066
Honor 10	53130
iphone 8	103668
Honor 102	109089
Honor 101	112567
Galaxy S9	113064
Galaxy S82	131640
iphone 72	150804

У ролі критерію сортування також можна використовувати складний вираз на основі стовпців:

```
42 •  select product_name, price, product_count
43    from product
44    order by product_count*price;
45
```

Result Grid | Filter Rows: | Export:

product_name	price	product_count
Galaxy S8	46066	1
Honor 10	26565	2
iphone 8	51834	2
Honor 102	12121	9
Honor 101	16081	7
Galaxy S9	56532	2
Galaxy S82	16455	8
iphone 72	12567	12

Сортування за спаданням.

За замовчуванням дані сортуються за зростанням, однак за допомогою оператора **DESC** можна задати сортування за спаданням:

```
42 •  select product_name, product_count
43    from product
44    order by product_count desc;
45
```

Result Grid | Filter Rows: | Export

product_name	product_count
Nokia 83	16
iphone 73	15
Nokia 82	13
iphone 72	12
iphone X3	12
iphone 83	12
Galaxy S93	12
Honor 103	12
Nokia 81	11
Galaxy S83	11
iphone X2	9

За замовчуванням замість **DESC** використовується оператор **ASC**, який сортує за зростанням. Для запитів, записаних нижче:

```
19 •  select product_name, product_count
20    from products
21    order by product_count asc;
```

або

```
43 •  select product_name, product_count
44    from product
45    order by product_count;
```

результат виконання буде однаковий:

Result Grid | Filter Rows: | Export: | Wrap Cell C

product_name	product_count
Galaxy 8	2
iphone X	3
iphone 8	3
Honor 10	3
Galaxy S9	6

Сортування за декількома стовпцями.

Для сортування відразу за декількома стовпцями їх всіх вказують через кому після оператора **ORDER BY**:

```
select product_name, price, manufacturer
from product
order by manufacturer, product_name;
```

Тут рядки спочатку сортуються за стовпцем manufacturer за зростанням. Далі якщо є два рядки, в яких стовпчик manufacturer має однакові значення, то вони сортуються за стовпцем product_name також за зростанням. Але за допомогою **ASC** і **DESC** можна окремо для різних стовпців задати сортування за зростанням і спаданням:

```
19  select product_name, price, manufacturer
20  from product
21  order by manufacturer, product_name desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

product_name	price	manufacturer
iphone X	71000	Apple
iphone 8	56000	Apple
Honor 10	26000	Huawei
Galaxy S9	56000	Samsung
Galaxy 8	46000	Samsung

Отримання діапазону рядків. Оператор LIMIT. Оператор дає змогу вибрати певну кількість рядків і має такий синтаксис:

LIMIT [offset,] rowcount;

Якщо оператору передається один параметр, то він вказує на кількість рядків, які необхідно вибрати. Якщо передається два параметра, то перший параметр задає зміщення відносно початку, тобто скільки рядків потрібно пропустити, а другий параметр також задає кількість рядків, які необхідно вибрати. Наприклад, виберемо три рядки:

```
19 •    select * from product
20      limit 3;
21
```

Result Grid | Filter Rows: | Edit: | Ex

id	product_name	manufacturer	product_count	price
1	iphone X	Apple	3	71000
2	iphone 8	Apple	3	56000
3	Galaxy S9	Samsung	6	56000
NULL	NULL	NULL	NULL	NULL

Тепер використаємо другий параметр і вкажемо змішення, з якого повинна виконуватись вибірка:

```
•    select * from product
     limit 2, 3;
```

У цьому випадку пропускаються два рядки, а три вибираються:

```
19 •    select * from product
20      limit 2, 3;
```

Result Grid | Filter Rows: | Edit: | Export/Imp

id	product_name	manufacturer	product_count	price
3	Galaxy S9	Samsung	6	56000
4	Galaxy 8	Samsung	2	46000
5	Honor 10	Huawei	3	26000
NULL	NULL	NULL	NULL	NULL

Як правило, оператор **LIMIT** використовують разом з **ORDER BY**:

```
19 •    select * from product
20      order by product_name
21      limit 2, 3;
```

Result Grid | Filter Rows: | Edit:

id	product_name	manufacturer	product_count	price
5	Honor 10	Huawei	3	26000
2	iphone 8	Apple	3	56000
1	iphone X	Apple	3	71000
NULL	NULL	NULL	NULL	NULL

Агрегатні функції.

Агрегатні функції обчислюють деякі скалярні значення в наборі рядків (стрічок). У MySQL є такі агрегатні функції:

AVG: обчислює середнє значення

SUM: обчислює суму значень

MIN: обчислює найменше значення

MAX: обчислює найбільше значення

COUNT: обчислює кількість рядків у запиті.

У всіх агрегатних функцій параметром є вираз, який слугує критерієм для обчислення значень. Найчастіше, у ролі виразу виступає назва стовпчика, над значеннями якого треба проводити обчислення.

Вираз у функціях **AVG** і **SUM** повинен бути числовим значенням (наприклад, стовпчик, який зберігає числове значення).

Вираз у функціях **MIN**, **MAX** і **COUNT** може бути числовим або стрічковим значенням або датою.

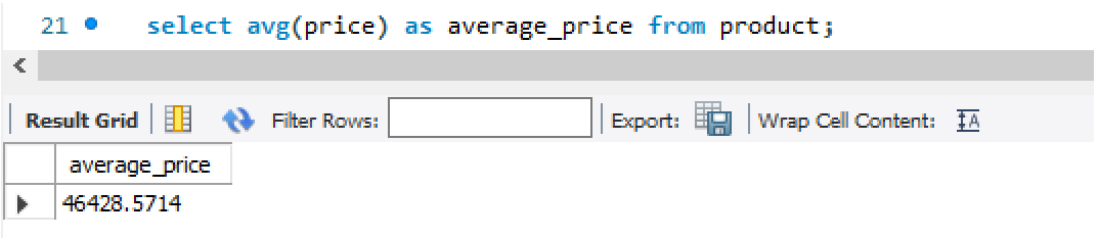
Усі агрегатні функції за винятком **COUNT (*)** ігнорують значення **NULL**.

AVG – повертає середнє значення із діапазону значень стовпця таблиці.

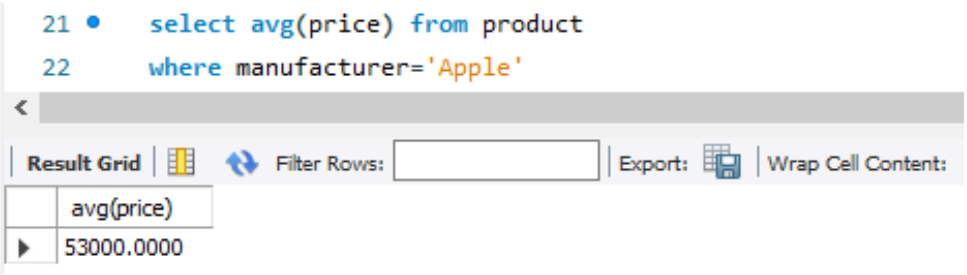
Наприклад, нехай є така таблиця товарів product:

```
4  • create table product (
5    id int auto_increment primary key,
6    product_name varchar(30) not null,
7    manufacturer varchar(30) not null,
8    product_count int default 0,
9    price decimal not null);
10
11 • insert into product  (product_name, manufacturer, product_count, price)
12   values
13   ('iphone X', 'Apple', 3, 71000),
14   ('iphone 8', 'Apple', 3, 56000),
15   ('iphone 7', 'Apple', 5, 32000),
16   ('Galaxy S9', 'Samsung', 6, 56000),
17   ('Galaxy 8', 'Samsung', 2, 46000),
18   ('Honor 10', 'Huawei', 3, 26000),
19   ('Nokia 8', 'HMD Global', 6, 38000);
```

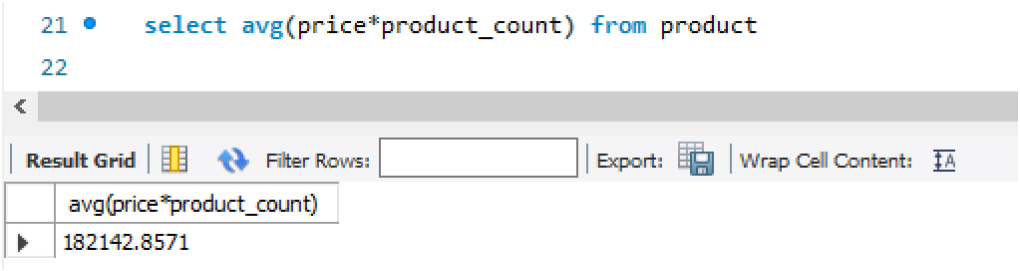
Знайдемо середню ціну товарів у таблиці. Для пошуку середнього значення у ролі виразу в функцію передається стовпчик Price. Для отримання потрібного значення встановлюється псевдонім Average_Price, хоча це не обов'язково.



На етапі вибірки можна застосовувати фільтрацію. Наприклад, знайдемо середню ціну для товарів певного виробника:



Також можна знаходити середнє значення для складніших виразів. Наприклад, знайдемо середню суму всіх товарів, враховуючи їх кількість:



COUNT. Обчислює кількість рядків у вибірці. Є дві форми цієї функції. Перша форма **COUNT (*)** рахує кількість рядків у вибірці:

```
21 •    select count(*) from product
22
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	count(*)
▶	7

Друга форма функції обчислює кількість рядків за певним стовпчиком, при цьому рядки зі значеннями **NULL** ігноруються:

```
    • select count(manufacturer) from product
```

MIN і MAX. Обчислюють мінімальне та максимальне значення за стовпцем, відповідно. Наприклад, знайдемо мінімальну і максимальну ціну серед товарів:

```
21 •    select min(price), max(price) from product
22
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	min(price)	max(price)
▶	26000	71000

Ці функції також ігнорують значення **NULL** і не враховують їх під час обчислень.

SUM. Обчислює суму значень стовпця. Наприклад, обчислимо загальну кількість товарів:

```
21 •    select sum(product_count) from product
22
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	sum(product_count)
▶	28

Також замість назви стовпця, можна задавати обчислювальний вираз. Наприклад, знайдемо загальну вартість всіх наявних товарів:

```
21 •    select sum(product_count*price) from product
22
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

sum(product_count*price)
1275000

ALL DISTINCT. За замовчуванням усі п’ять функцій враховують всі рядки вибірки для обчислення результату. Але вибірка може містити значення, які повторюються. Якщо необхідно виконати обчислення тільки над унікальними значеннями, виключивши з набору значень ті дані, що повторюються, то для цього застосовують оператор **DISTINCT**.

```
21 •    select count(distinct manufacturer) from product
22
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

count(distinct manufacturer)
4

За замовчування замість **DISTINCT** застосовується оператор **ALL**, який вибирає всі рядки:

```
21 •    select count(manufacturer) from product
22
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

count(all manufacturer)
7

У цьому випадку бачимо, що виробники можуть повторюватися в таблиці, оскільки деякі товари можуть мати одних і тих самих виробників. Тому, щоб

визначити кількість унікальних виробників, необхідно використовувати оператор **DISTINCT**. Якщо така задача не ставиться, можна нічого не вказувати.

Комбіновані функції. Об'єднаємо декілька функцій:

```
21 •  select count(*) as product_count,
22    sum(product_count) as totalcount,
23    min(price) as minprice,
24    max(price) as maxprice,
25    avg(price) as avgprice
26    from product
27
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_count	totalcount	minprice	maxprice	avgprice
7	28	26000	71000	46428.5714

Групування даних. Оператори **GROUP BY** і **HAVING** дають змогу згрупувати дані. Їх використовують у рамках команди **SELECT**:

SELECT стовпці

FROM таблиця

[**WHERE** умова_фільтрації_рядків]

[**GROUP BY** стовпці_для_групування]

[**HAVING** умова_фільтрації_груп]

[**ORDER BY** стовпці_для_сортування]

GROUP BY. Задає *правило групування рядків*. Наприклад, згрупуємо товар за виробником:

```
72 •    select manufacturer, count(*)  as models_count
73      from product
74      group by manufacturer
75
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

manufacturer	models_count
Apple	3
Samsung	2
Huawei	1
HMD Global	1

Перший стовпчик у виразі **SELECT** – manufacturer представляє назву групи, а другий – models_count представляє результат функції **COUNT**, яка визначає кількість рядків у групі.

І якщо у виразі **SELECT** проводиться вибірка за одним або кількома стовпцями і також використовуються агрегатні функції, то необхідно використати оператор **GROUP BY**.

Оператор **GROUP BY** може виконувати групування за множиною стовпців. Так, додамо групування за кількістю товарів:

```
72 •    select manufacturer, product_count, count(*)  as models_count
73      from product
74      group by manufacturer, product_count
75
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

manufacturer	product_count	models_count
Apple	3	2
Apple	5	1
Samsung	6	1
Samsung	2	1
Huawei	3	1

Необхідно враховувати, що вираз **GROUP BY** повинен іти після виразу WHERE, але до виразу **ORDER BY**:

```
72 •  select manufacturer, count(*)  as models_count
73    from product
74    where price>30000
75    group by manufacturer
76    order by models_count desc
77
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

manufacturer	models_count
Apple	3
Samsung	2
HMD Global	1

Фільтрація груп. **HAVING**. Дає змогу виконувати фільтрацію груп, тобто задає, які групи будуть включені у вихідний результат.

Використання **HAVING** багато у чому аналогічне, до застосування **WHERE**. Однак **WHERE** застосовують для фільтрації рядків, а **HAVING** – для фільтрації груп. Наприклад, знайдемо всі групи товарів за виробниками, для яких визначено більше однієї моделі:

```
72 •  select manufacturer, count(*)  as models_count
73    from product
74    group by manufacturer
75    having count(*)>1
76
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

manufacturer	models_count
Apple	3
Samsung	2

В одній команді також можна поєднувати оператори **HAVING** та **WHERE**:

```
72 •   select manufacturer, count(*)  as models_count
73     from product
74     where price*product_count>80000
75     group by manufacturer
76     having count(*)>1
77
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

manufacturer	models_count
Apple	3
Samsung	2

Тобто у цьому випадку спочатку фільтруються рядки: вибираються товари, загальна вартість яких більше 80000. Потім вибрані товари групуються за виробниками. А далі фільтруються уже групи – вибираються ті, які мають більше однієї моделі. Якщо при цьому необхідно виконати сортування, то вираз **ORDER BY** іде після виразу **HAVING**:

```
72 •   select manufacturer, count(*)  as models, sum(product_count) as units
73     from product
74     where price*product_count>80000
75     group by manufacturer
76     having sum(product_count)>2
77     order by units desc
78
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

manufacturer	models	units
Apple	3	11
Samsung	2	8
HMD Global	1	6

Тут групування йде за виробниками, також вибирається кількість моделей кожного виробника (models) і загальна кількість всіх товарів за цими всіма моделями (units). Врешті групи сортуються за кількістю товарів за спаданням.

Представлення. Оператор VIEW.

У SQL таблицю, яку створюють за допомогою запиту **CREATE TABLE**, називають ***базовою таблицею***. Її структура та дані зберігаються операційною системою.

Технічно результатом кожного запиту SELECT також є таблиця. Таку таблицю називають ***похідною***.

Представлення – це не що інше, як запити SQL, які зберігаються в базі даних під певним ім'ям. Насправді це таблиця, яка була згенерована за допомогою попередньо визначеного SQL-запиту.

Представлення може містити всі рядки даної таблиці або частини, яка нас цікавить. Його можна створити на основі однієї або кількох таблиць. Все залежить від самого SQL-запиту та ефекту, якого ми хочемо досягти.

Представлення – це тип віртуальних таблиць, над якими можна виконувати низку операцій, наприклад:

• структурування даних у доступній формі;

• обмеження (представлених) даних лише необхідними;

• об'єднання результатів з різних таблиць, які можуть бути основою для формування звітів.

Основною командою для створення представлень є **CREATE VIEW**.

Основний запит із використанням наведеної вище команди має такий вигляд:

CREATE VIEW назва_подання **AS**
SELECT стовп1, стовп2, …, стовпN
FROM назва_табл
WHERE умова

Можна створювати складні запити, об'єднувати багато таблиць або використовувати підзапити, як у випадку звичайного **SELECT**.

Нехай у базі даних маємо таблицю співробітників компанії employee:

```
create table employee
(
empNum int auto_increment primary key,
last_name varchar(20) not null,
first_name varchar(15) not null,
e_mail varchar(60),
deptNum int,
salary int,
start_date date);
```

Запит на вибірку інформації про ім'я і прізвище працівника, номер відділу та початок роботи матиме вигляд:

```
select empNum, last_name, first_name, deptNum, start_date
from employee;
```

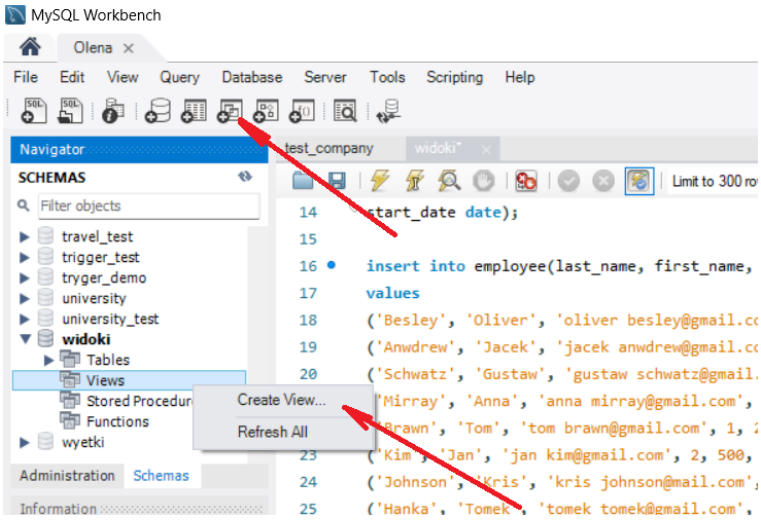
У результаті виконання запиту отримаємо таблицю:

Result Grid | Filter Rows: | Edit:

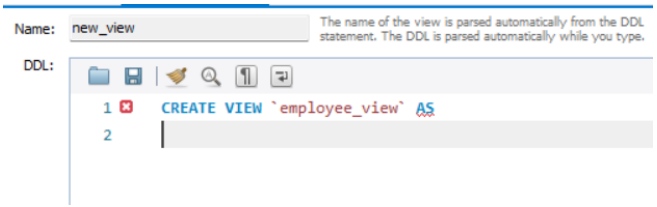
empNum	last_name	first_name	deptNum	start_date
1	Besley	Oliver	3	2021-01-25
2	Anwdrew	Jacek	1	2020-01-25
3	Schwatz	Gustaw	3	2021-02-01
4	Mirray	Anna	3	2019-01-25
5	Brawn	Tom	1	2020-06-15
6	Kim	Jan	2	2022-01-25
7	Johnson	Kris	2	2021-10-01
8	Hanka	Tomek	1	2021-01-01
9	Murrai	Keith	2	2022-02-01
10	Lutra	Pavel	3	2021-01-25

Наступного разу, якщо ви захочете отримати ту саму інформацію, вам потрібно буде знову надіслати той самий запит. Один зі способів уникнути повторного звертання до бази даних – це зберегти запит у файлі .txt або .sql, щоб пізніше можна було відкрити та виконати його у MySQL Workbench або будь-якому іншому клієнтському інструменті MySQL. Найкращий спосіб для цього – зберегти запит на сервері бази даних і надати йому ім'я, тобто створити представлення. Щоб

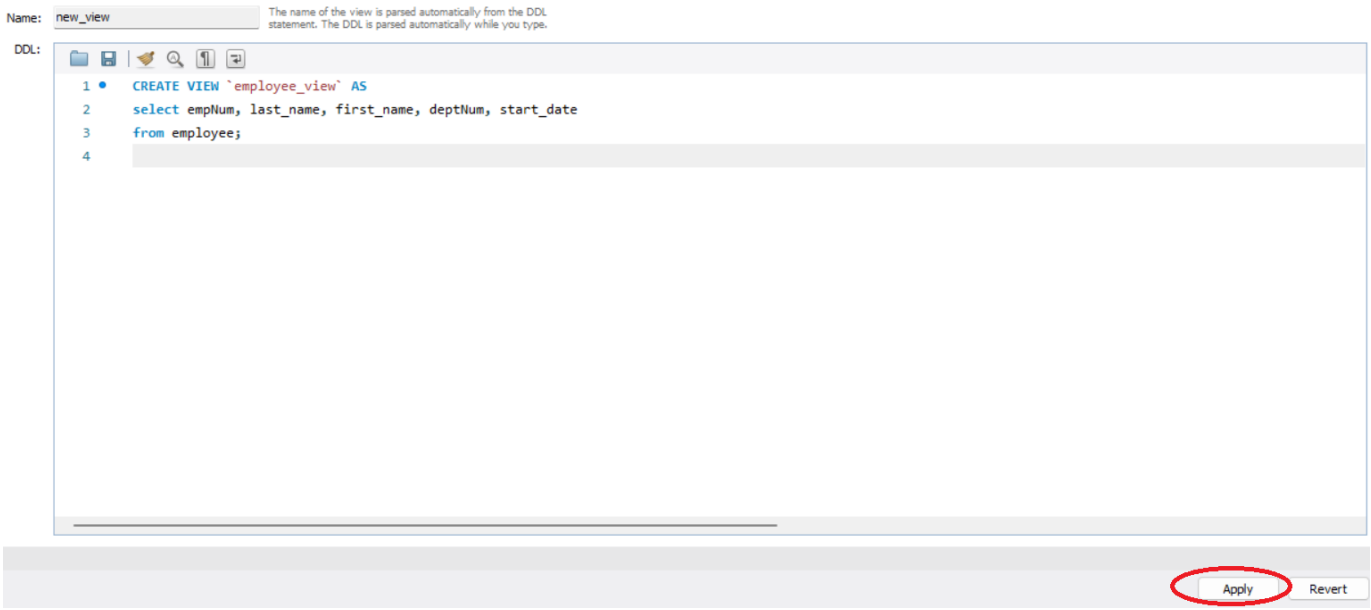
створити представлення, потрібно правою кнопкою миші вибирати Views → Create VIEW або піктограму вказану стрілкою:



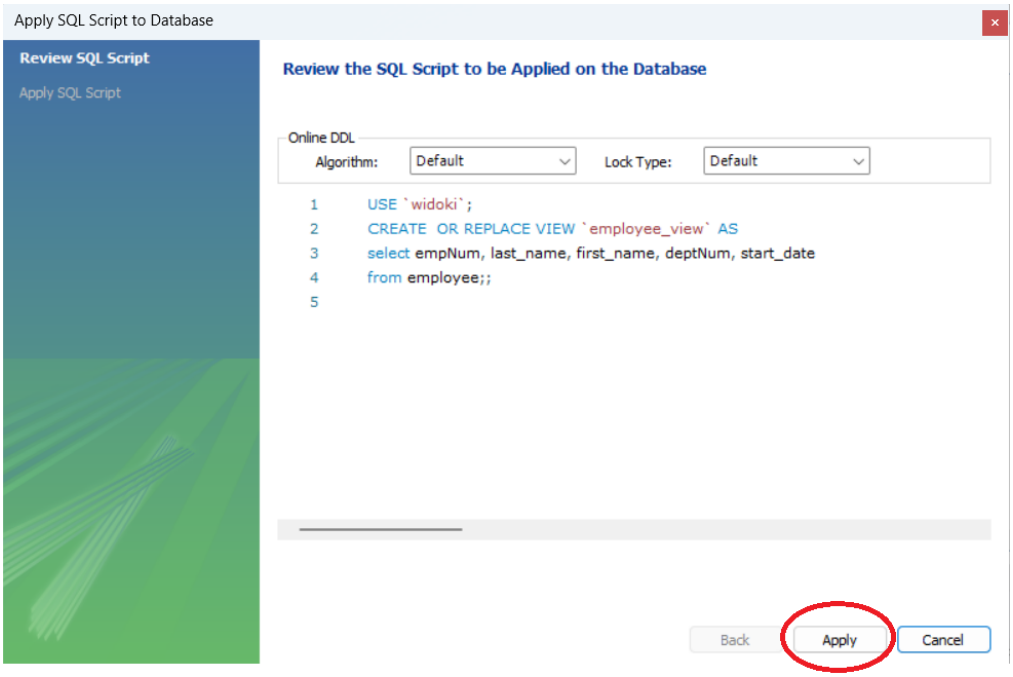
Відкриється вікно для створення нового представлення і введіть його назву:



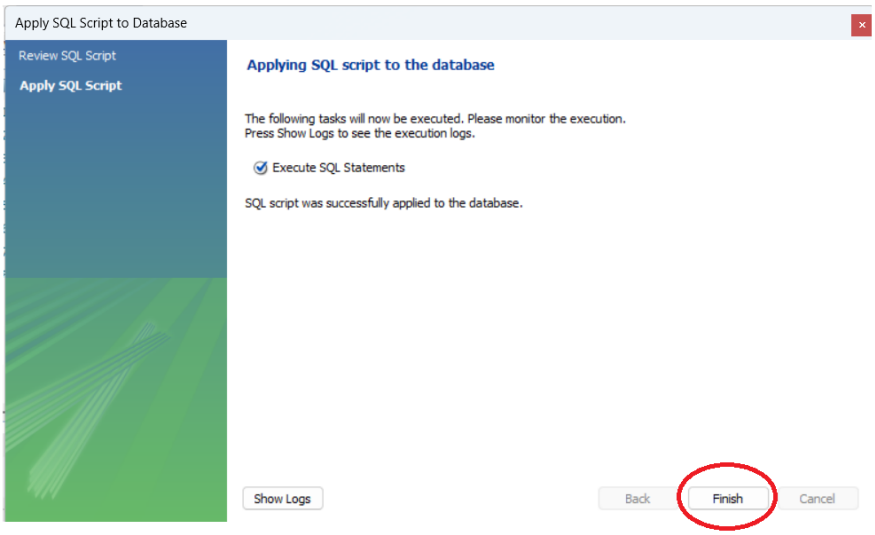
Після введення назви представлення employee_view та запиту до бази даних отримаємо вікно:



Далі потрібно натиснути кнопку Apply в правому нижньому кутку вікна. Відкриється наступне вікно, в якому можна переглянути скрипт і застосувати його до бази, натиснувши знову кнопку Apply.



З’явиться вікно про приєднання процедури до бази даних, для завершення процесу необхідно натиснути кнопку Finish:



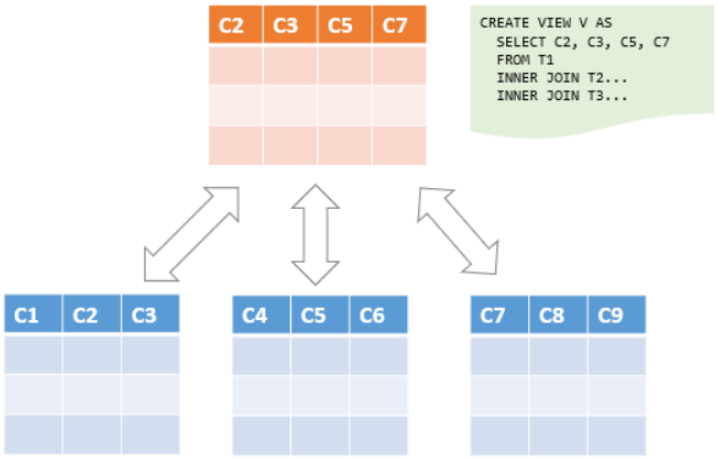
Тепер можна посилатися на представлення як на таблицю в операторах SQL. Наприклад, можна запитувати дані з представлення employee_view за допомогою оператора **SELECT**:

```
select * from employee_view;
```

Як бачите, синтаксис набагато простіший.

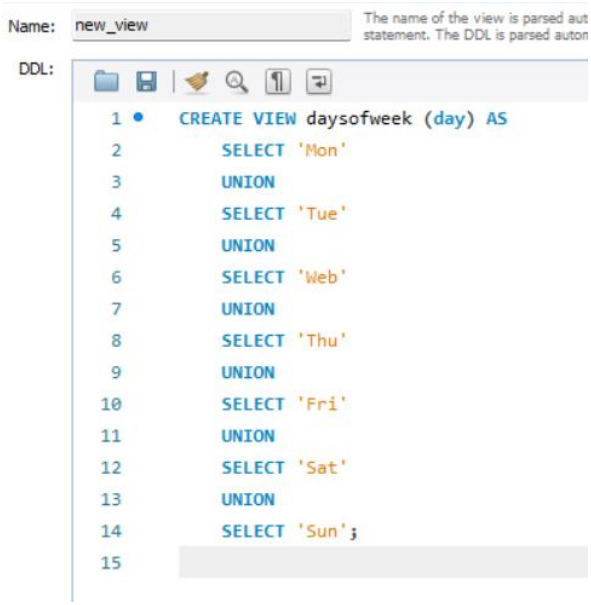
Зауважте, що представлення фізично не зберігає дані. Коли ви виконуєте оператор **SELECT** для представлення, MySQL виконує базовий запит, зазначений у визначенні представлення, і повертає набір результатів. З цієї причини представлення називають віртуальною таблицею.

MySQL дає можливість створювати представлення на основі оператора **SELECT**, який отримує дані з однієї або кількох таблиць. На цьому малюнку показано подання на основі стовпців кількох таблиць:



Крім того, MySQL навіть дає змогу створювати представлення, яке не посилається на жодну таблицю. Хоча такий вид подання не часто зустрічається на практиці.

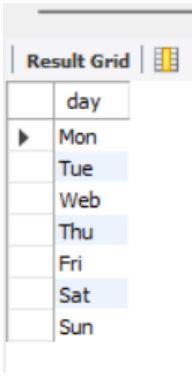
Наприклад, можна створити представлення під назвою daysofweek, яке повертає 7 днів тижня, виконавши такий запит



Дані з представлення daysofweek можна запитувати таким чином:

```
select * from daysofweek;
```

Результат виконання запиту:



3. Типові контрольні запитання

1. За допомогою якого запиту можна вивести інформацію з певної таблиці БД?
2. Який запит дає змогу вибрати унікальні значення виробників із БД?
3. Назвіть оператори фільтрації та охарактеризуйте їх застосування?
4. Запишіть синтаксис оператора сортування стовпців.
5. Як задати псевдонім стовпця?
6. Символи та їх зміст в операторах пошуку за шаблоном.

7. Який запит дає змогу сортувати стовпці за спаданням?
8. Запишіть запит сортування за декількома стовпцями у таблиці БД.
9. Які агрегатні функції використовують у запитах?
10. За допомогою яких операторів можна групувати дані?
11. Як здійснити фільтрацію груп?
12. Для чого використовують представлення?
13. Що таке віртуальна таблиця?

4. Завдання для виконання лабораторної роботи

Для бази даних за індивідуальним завданням виконати *завдання 1–7*:

Завдання 1. Використовуючи оператори **SELECT**, **FROM**, **LIMIT**, **DISTINCT**, **IS NULL** та інші, написати **5 скриптів** для створення запитів простої вибірки з таблиць бази даних.

Завдання 2. Використовуючи оператори **ORDER BY**, **DESC**, написати **3 скрипти** для здійснення сортування даних під час створення запитів до бази даних.

Завдання 3. Використовуючи оператори **IN**, **BETWEEN**, **LIKE**, **REGEXP** написати **5 скриптів** для здійснення фільтрації даних при створенні запитів до бази даних.

Завдання 4. Використовуючи агрегатні функції, написати **5 скриптів** для створення запитів різного типу до бази даних.

Завдання 5. Використовуючи оператори **GROUP BY** і **HAVING**, написати **3 скрипти** для групування вибраних даних під час створення аналітичних та підсумкових запитів до бази даних.

Завдання 6. Використовуючи оператор **VIEW** створити **2 представлення** для висвітлення інформації із таблиць бази даних.

Зміст звіту:

1. Титульний лист
2. Завдання 1.
3. Представити структуру скриптів запитів, написаних до бази даних обраної предметної області, та скріншоти результатів їх виконання запитів згідно з завданням 1 до лабораторної роботи.
4. Завдання 2….
5. ….
6. Висновки.

5. Список літератури

1. Мулеса О. Ю. Інформаційні системи та реляційні бази даних. Ужгород, 2018. 118 с.

2. Павельчак А. Г., Самотий В. В., Дзелендзяк У. Ю. Бази даних та знань. Львів, 2011. 115 с.

3. Вступ у MySQL. https://metanit.com/sql/mysql/2.5.php

4. https://coderlessons.com/tutorials/bazy-dannykh/osnovy-sql/7-operator-select-2

5. https://dev.mysql.com/doc/workbench/en/wb-sql-editor-query-panel.html